Warm Up: Efficient Semantic Segmentation

AUTHOR
Christian Osendorfer

AFFILIATION
Hochschule Landshut

PUBLISHED
February 7, 2026

MODIFIED
March 19, 2026

In this warm-up assignment you will implement and analyze an efficient semantic segmentation model built around a **MobileNetV2-style encoder** and a **lightweight decoder**. The goal is to understand how architectural choices and scaling affect the tradeoff between segmentation quality and computational cost — a different set of concerns than image classification.

We are building a segmentation model for **Cityscapes** at **256×512** resolution with **19 classes** (standard train IDs). You'll design the architecture, implement the full pipeline, train a baseline, and eventually run a small scaling study.

Dataset

Use the Cityscapes dataset (fine annotations only). Register and download it manually.

```
from torchvision.datasets import Cityscapes

train_ds = Cityscapes(root="data", split="train", mode="fine", target_type="semantic")
val_ds   = Cityscapes(root="data", split="val",   mode="fine", target_type="semantic")
```

Convert labels to train IDs. Mask invalid labels as `-1` for loss computation.

Data Pipeline

Implement a preprocessing pipeline that resizes images to 256×512 (masks with nearest-neighbor interpolation), normalizes images, and returns `(image, mask)` tensors with valid class indices. Verify image/mask alignment and class value ranges before training.

Modeling

Your model follows the U-Net architecture: a contracting encoder that builds a hierarchy of feature representations at progressively lower resolutions, paired with an expanding decoder that recovers spatial detail by reintegrating features from the encoder via skip connections. The encoder is modeled after MobileNetV2, whose inverted residual blocks offer a favorable accuracy-to-compute tradeoff suitable for dense prediction tasks.

Encoder

The core building block is the **inverted residual block**: It expands channels via a 1×1 convolution, applies a depthwise 3×3 convolution for spatial processing, and then projects back to a lower-dimensional output via a linear 1×1 convolution. A residual connection is used when the stride is 1 and input and output channels match.

This design, which expands before the depthwise convolution and projects back afterward, is the inverse of the classical ResNet bottleneck. In combination with depthwise separable convolutions and linear bottlenecks, it enables the high efficiency of MobileNetV2.

Importantly, the efficiency does not arise from the inversion alone, but from the combination of three factors. First, the spatial convolution is performed depthwise, reducing cost from $O(k^2C^2)$ to $O(k^2C)$. Second, the expansion step allows nonlinear processing in a higher-dimensional space, improving expressivity. Third, the projection step returns to a narrow bottleneck, which keeps memory usage and residual connections inexpensive. Without depthwise separable convolutions, the inversion by itself would not yield comparable efficiency gains.

For reference, the classical ResNet bottleneck follows the sequence: a 1×1 convolution reduces channels (C C/4), a 3×3 convolution operates in this reduced space, and a final 1×1 convolution expands back $C/4 \to C$. Thus, *inverted* refers to reversing the order of channel scaling: ResNet compresses and then expands, whereas MobileNetV2 expands and then compresses.

Stack these blocks into multiple encoder stages, each downsampling by a factor of two, so that the encoder spans resolutions from $\frac{1}{2}$ down to (e.g.) $\frac{1}{32}$ of the input. Each stage produces a feature map that will later serve as a skip connection. Your final report (see below) should include a table listing the spatial resolution, channel count, and number of blocks at each stage.

Decoder

The decoder mirrors the encoder, recovering resolution stage by stage. At each step, the feature map is upsampled bilinearly, concatenated with the corresponding encoder feature map, compressed with a 1×1 convolution to control channel count, and refined with two 3×3 convolutions. You may replace the standard convolutions with depthwise-separable variants to reduce compute. If you prefer addition over concatenation as your fusion strategy, you will need a projection to align channel dimensions — discuss the tradeoffs either way.

Full Model

The encoder and decoder are composed into a single module parameterized by `width_mult` and `depth_mult`, which uniformly scale channel counts and block counts respectively. These multipliers are the basis of the scaling study (see below). The model outputs raw logits of shape `(B, 19, H, W)`; apply `F.cross_entropy` with `ignore_index=-1` to mask invalid labels during training.

Training

Implement training and validation loops with checkpoint saving and TensorBoard logging (loss, mIoU, accuracy, and periodic prediction visualizations). Report **pixel accuracy** and **mean IoU** on the validation set.

Scaling Study

Run a small compound scaling study. Train and compare at least four models:

Model	`width_mult`	`depth_mult`	Resolution
Baseline	1.0	1.0	256×512
Wide	> 1.0	1.0	256×512
Deep	1.0	> 1.0	256×512
Combined	> 1.0	> 1.0	256×512 or 384×768

For each model report: parameter count, MACs (use `profile_macs` from `torchprofile`), mIoU, and inference time. Explore what other metrics torchprofile exposes beyond MACs, and note whether any results differ when the model is profiled on GPU vs. CPU.

Ablation

Perform *one* controlled ablation — e.g., concatenation vs. addition fusion, standard vs. depthwise decoder convolutions, or decoder depth — with all other variables fixed.

Evaluation Script

Write a standalone script that loads a checkpoint, run inference on a given folder, can properly evaluate when ground truth is available, reports metrics, and visualizes predictions alongside ground truth.

Deliverables

1. **Code** — full implementation, training script, evaluation script
2. **Report (3-5 pages)** — architecture description, training setup, results table, qualitative examples, and answers to the questions below
3. **Checkpoints** — baseline and one scaled variant. Your inference code can run these checkpoints on images provided in a folder.

Report Questions

1. How is your encoder structured, and why?
2. What decoder design did you choose and what are its tradeoffs?
3. Which scaling axis (width, depth, resolution) yielded the most gain?
4. What was the primary bottleneck — compute, memory, or architecture?
5. What failure modes did you observe?

Optional Extensions

- **Squeeze-and-Excitation (SE) blocks** — Insert a channel attention mechanism after depthwise convolutions to let the network recalibrate feature responses. Adds minimal parameters but often improves accuracy. (Hu et al., 2018)
- **Dice loss** — An alternative to cross-entropy that directly optimizes overlap between predicted and ground-truth masks, which can help with class imbalance. Often combined with cross-entropy as a joint loss. (Milletari et al., 2016)
- **Per-class IoU analysis** — Break down your mIoU into per-class scores and examine which semantic categories your model struggles with most. Cross-reference against class frequency to distinguish rare-class failure from architectural limitations. (Cityscapes benchmark)

🎉 ✨ You did it! Congratulations.